Hacking the Internet of Things: TomTom Runner Smartwatch

Luis Grangeia | @lgrangeia

September 2015
Confraria de Segurança da Informação

Overview

- Introduction
- Motivations
- Attack surface
- Analysis
- Exploitation
- Next steps

Introduction

Introduction

- Saw these guys…

Introduction

- … doing this:

"Remote Exploitation of an Unaltered Passenger Vehicle" - Charlie Miller and Chris Valasek

Introduction

- **“I want to do that!”**
- Lots of uncharted territory in IoT hacking
- ‘Embedded systems’ is not my forté…
  - Getting to learn new stuff: **WIN**

Not done yet!

Motivations

Motivations

- To get started at reverse engineering and IoT hacking
- IoT == Embedded ARM devices connected to the internet
- ARM is the trend in mobile computing
- **IoT Reversing actually seems easier than x86 reversing**
  - less exploit mitigations
- “Bending” closed hardware to my will is way cooler than hacking a general purpose system

Target: TomTom Runner

- Running Sportswatch
- Bluetooth LE + GPS
- Syncs with PC via USB, smartwatch via Bluetooth
- Closed system

Why the TomTom Runner?

- Other targets were considered:
  - SPoQ SQ-100:
    - AVR Microcontroller (not ARM)
    - Firmware is not encrypted
    - No Bluetooth (ANT operates on same frequency though…)
    - Deemed **Not Cool Enough™**

Why the TomTom Runner?

- Other targets were considered:
  - Volkswagen RCD 510:
    - My car's head unit
    - Firmware is encrypted
    - Expensive
    - Architecture unknown
    - Attached to car
    - (maybe later ☺)

TomTom Runner
External hardware

- Dot matrix LCD
- Decent battery (lasts weeks)
- USB 2.0 Interface
- Bluetooth + GPS
- Beeper + Buzzer
- Four-way D-PAD

Rules of Engagement

- Rule 1: No physical tampering!
  - it's waterproof!
- Rule 2: Do not brick it!
  - Almost did, several times…
- **Rule 3: There are no rules**

Attack Surface

Attack Surface

- USB Interface
- Bluetooth
- User Interface

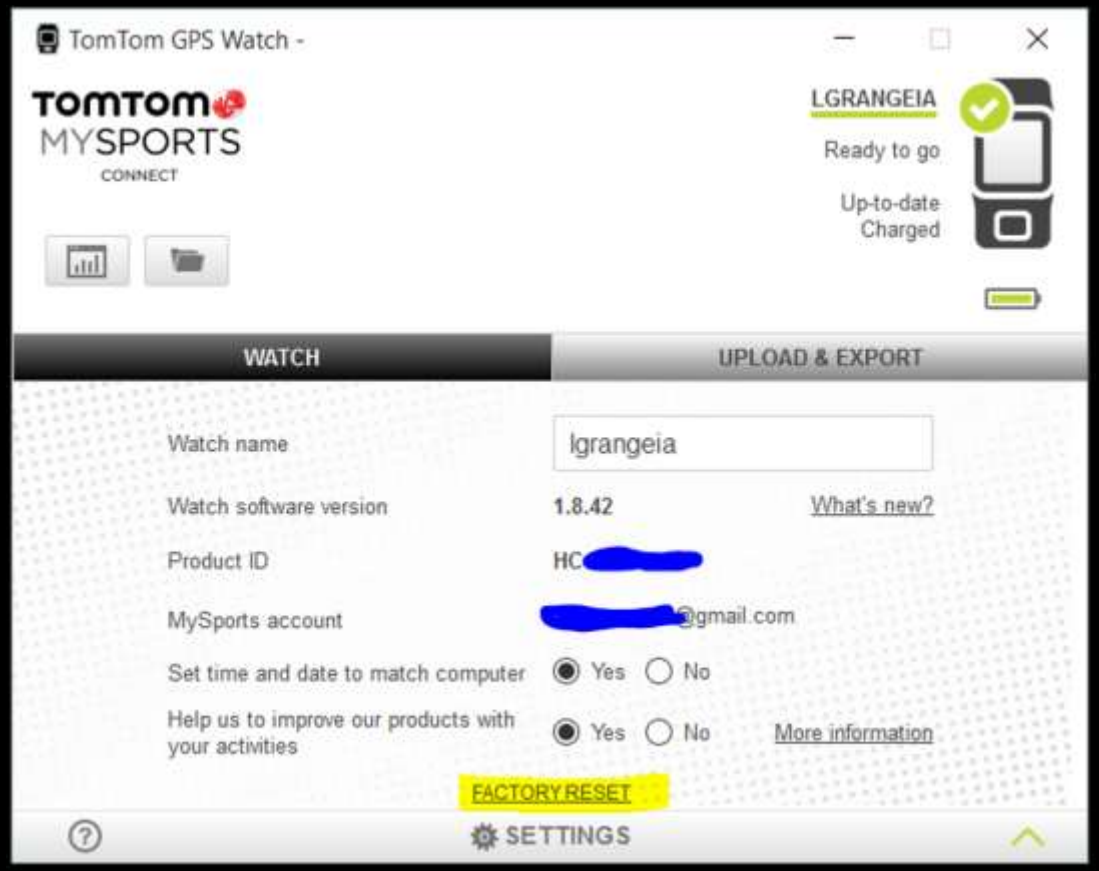
Attack Surface

Analysis

USB, Firmware, “Dissection”

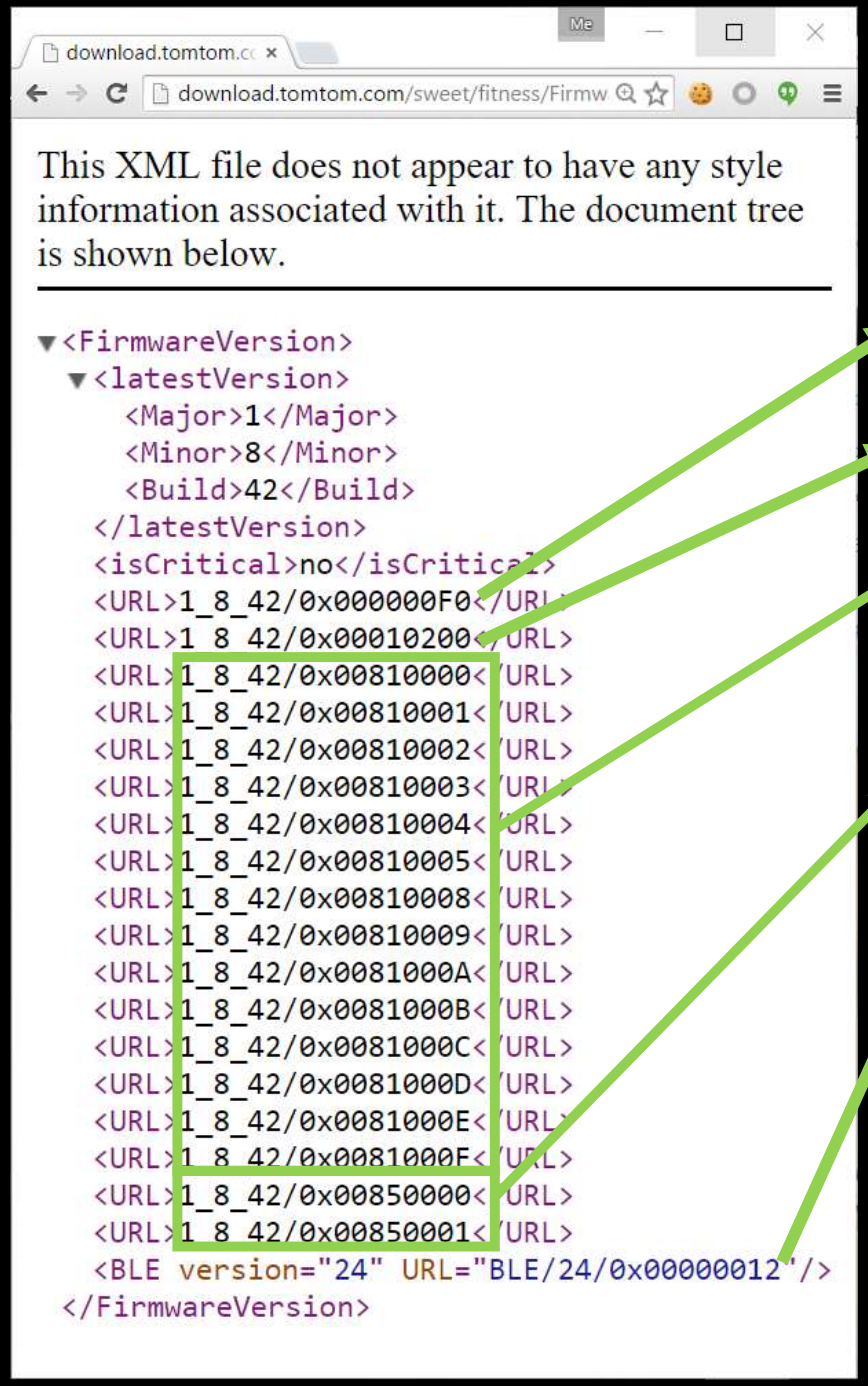
Attack Surface: USB

- Obtain Firmware via forced upgrade (Tomtom Mysports Connect)



Attack Surface: USB

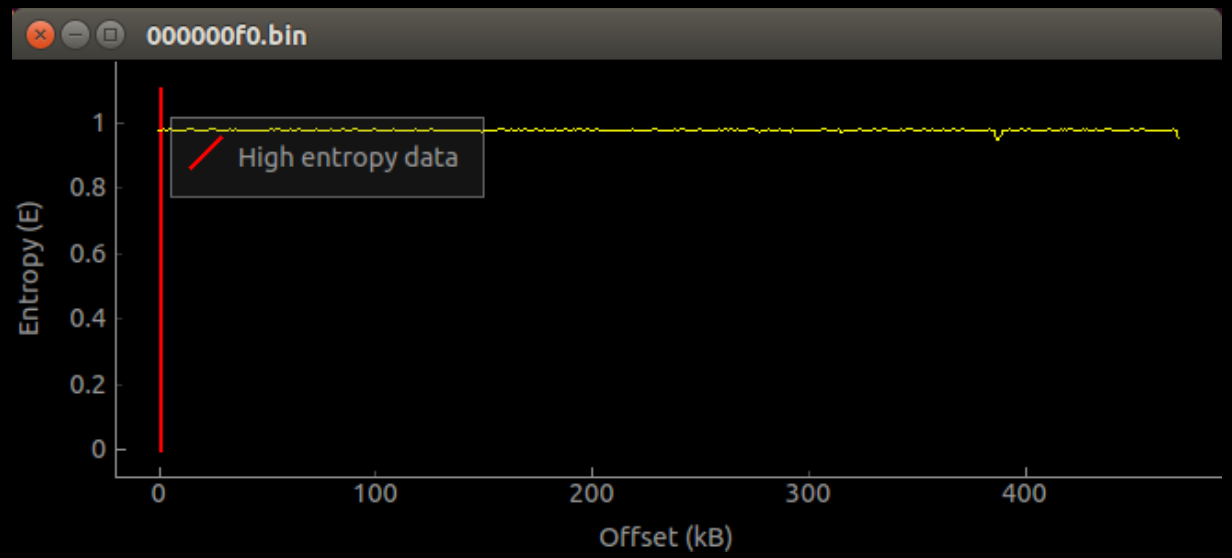
- Better alternative: Linux!
  - Linux TomTom GPS Watch Utilities
  - github.com/ryanbinns/ttwatch
- Use the source!
- Firmware is available here:
  - download.tomtom.com/sweet/fitness/Firmware/E9030000/FirmwareVersionConfigV2.xml



- Main Firmware file
- Firmware for the GPS Module
- Language resource files (eng / ger/ por / etc.)
- Manifest files (configuration settings)
- Firmware for the BLE Module
- **Note to TomTom: Use SSL!**

Reversing the main firmware

- Binwalk:




```
lgrangeia@vostok: ~/tomtom/Firmware/1.8.42
lgrangeia@vostok:~/tomtom/Firmware/1.8.42$ binwalk -BEH 000000f0.bin

DECIMAL       HEXADECIMAL     HEURISTIC ENTROPY ANALYSIS
--------------------------------------------------------------------------------
1024          0x400           High entropy data, best guess: encrypted, size: 470544, 0 low entropy blocks


DECIMAL       HEXADECIMAL     HEURISTIC ENTROPY ANALYSIS
--------------------------------------------------------------------------------
```

Reversing the main firmware

- Comparing different versions of the same file (vbindiff)

```
lgrangeia@vostok: ~/tomtom/Firmware
1.8.35/0x000000F0
0000 0000: 83 66 4B 77 9E E2 D6 94  9A BE 8D 13 56 72 CB B2  .fKw.... ....Vr..
0000 0010: 49 E8 BA 62 49 E5 4F F5  F5 C4 FE 6C A2 4C D4 BE  I..bI.O. ...l.L..
0000 0020: 80 CF 75 97 B5 67 04 F3  D7 22 8A A6 A0 0A 4A 6A  ..u..g.. ."....Jj
0000 0030: 49 8C 53 D9 7B 32 AE E0  95 0D 13 25 27 3A 4F 12  I.S.{2.. ...%':O.
0000 0040: 3B 09 5C 19 12 95 CF 19  0A 10 67 50 D0 FF 6D 97  ;.\..... ..gP..m.
0000 0050: A8 32 29 28 9F B5 BE 29  6B A0 31 1F B7 2D E1 5D  .2)+...) k.i..-.]
0000 0060: 85 58 18 8D 82 34 BB A9  7B 78 A4 36 01 60 E9 22  .X...4.. {x.6.`."
0000 0070: 31 50 AE 7D CD 01 CF C7  46 58 0E C1 7C DB 27 18  1P.}.... FX..|.'.
0000 0080: 8D 41 F3 A5 55 C2 CB D6  FB 95 36 57 0A F6 0C B2  .A..U... ..6W....
0000 0090: E6 8D 1D 94 2A 94 D0 BA  B9 CA F4 57 8C 3D 2D 5B  ....*... ...W.=-[
0000 00A0: E9 AF DC 69 77 7A CA 00  AC 3D 5E AB CB 92 46 F5  ...iwz.. .=^...F.
0000 00B0: A8 7A BF E9 01 C7 B8 B4  16 D3 BF C9 7A 99 2A 25  .z...... ....z.*%
0000 00C0: AB EC D2 D3 85 BC 8D F7  CB F1 3E FA E7 87 92 D0  ........ ..>.....
0000 00D0: 29 F3 70 09 7D 93 23 05  CE F3 6B D1 0B 7A B8 73  ).p.}.#. ..k..z.s
0000 00E0: 4B 2E 51 EA 42 B7 55 1E  BF A3 3E 39 5A 3B 39 A7  K.Q.B.U. ..>9Z;9.
0000 00F0: 5E EC A0 2E 29 E0 2E 69  C5 99 7F E9 C3 41 E7 FA  ^...)..i .....A..
1.8.25/0x000000F0
0000 0000: DD 0E 2B A8 7B 1C 60 65  AC 95 76 F8 2B ED F9 E3  ..+.{.`e ..v.+...
0000 0010: 77 C2 B5 02 64 0C BF 26  7B 13 6B 57 27 F6 74 7D  w...d..& {.kW'.tp
0000 0020: 80 CF 75 97 B5 67 04 F3  D7 22 8A A6 A0 0A 4A 6A  ..u..g.. ."....Jj
0000 0030: 49 8C 53 D9 7B 32 AE E0  95 0D 13 25 27 3A 4F 12  I.S.{2.. ...%':O.
0000 0040: 3B 09 5C 19 12 95 CF 19  0A 10 67 50 D0 FF 6D 97  ;.\..... ..gP..m.
0000 0050: B3 32 B2 6C BC 1B FE D7  62 6D 46 48 56 19 94 57  .2.l.... bmFHV..W
0000 0060: 85 58 18 8D 82 34 BB A9  7B 78 A4 36 01 60 E9 22  .X...4.. {x.6.`."
0000 0070: 1F 47 AA DB 9D 22 19 D0  DB 56 A8 6A FA EF E1 8B  .G..."... .V.j....
0000 0080: B2 C0 46 0E 5D B0 64 40  9A F8 EF 55 71 44 F7 E4  ..F.].d@ ...UqD..
0000 0090: E6 8D 1D 94 2A 94 D0 BA  B9 CA F4 57 8C 3D 2D 5B  ....*... ...W.=-[
0000 00A0: 42 D0 24 D1 E0 72 11 57  8F 1C 44 57 EA B3 E9 E1  B.$..r.W ..DW....
0000 00B0: A8 7A BF E9 01 C7 B8 B4  16 D3 BF C9 7A 99 2A 25  .z...... ....z.*%
0000 00C0: AB EC D2 D3 85 BC 8D F7  CB F1 3E FA E7 87 92 D0  ........ ..>.....
0000 00D0: 70 A0 D2 AF C0 0B 90 A0  54 67 5C 4B 10 C8 8F 44  p....... Tg\K..oD
0000 00E0: B0 40 AF 10 46 39 4B 61  73 6E DB C8 7B FD 09 E9  .M..F9Ha sn..{...
0000 00F0: 5E EC A0 2E 29 E0 2E 69  C5 99 7F E9 C3 41 E7 FA  ^...)..i .....A..

Arrow keys move   F find       RET next difference  ESC quit   T move top
C ASCII/EBCDIC    E edit file   G goto position       Q quit   B move bottom
```

Reversing the main firmware

- Best guess:
  - AES Encrypted, ECB mode
  - MAC Validated (ECB block shuffling fails)
- This hypothesis is reinforced by a document from **Atmel** (more on that in a sec)

Other firmware files

- BLE Firmware is not encrypted:
  - Flashed by the main firmware
  - MD5 Validated:

00013001.bin

```
2015-09-13 15:24:50 (UTC) Updating BLE
2015-09-13 15:24:51 (UTC) Current ver.024 Update ver.024
2015-09-13 15:24:51 (UTC) MD5 is correct!
2015-09-13 15:24:51 (UTC) Programming
2015-09-13 15:26:21 (UTC) Deleting file
2015-09-13 15:26:34 (UTC) Update done!
```

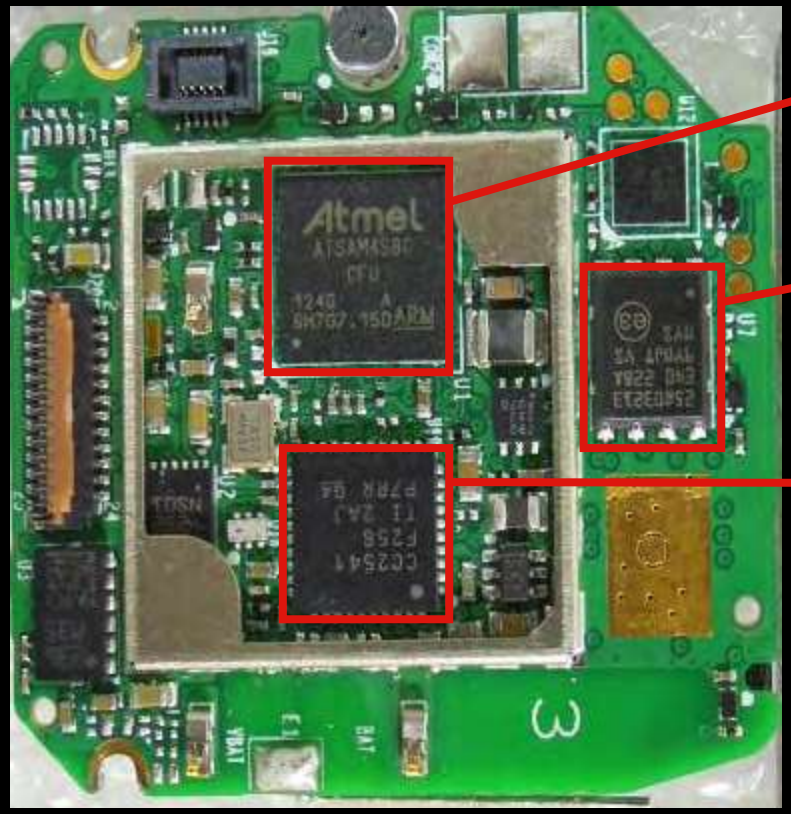
41 characters selected | Tab Size: 4 | Plain Text

- GPS Firmware also not encrypted

“opening” the device

- FCC == X-Ray Vision!
  - Every RF device sold in USA opened, photographed, tested
  - Searchable and available for everyone
- Search:
  - http://fcc.io
  - thanks @dominicgs !

“opening” the device



- Atmel ATSAM4S8C
  - Main “CPU” (MCU):
- Micron N25Q032A13ESC40F
  - Serial flash memory (4MB)
- Texas Instruments CC2541:
  - Bluetooth Module
- CSR SiRF starV 5e GNSS
  - GPS Module (off screen)

Google helps:
“A look inside the TomTom GPS Watch”
http://www.eetasia.com/ART_8800713547_1034362_NP_cb9c106d.HTM

USB Protocol Reversing

- Device resets when USB is connected / disconnected
- Device not usable while in USB mode
- Most of the USB communication is Reading / Writing files to EEPROM
  - **Language files**
  - Race / Exercise files
  - Preferences files
  - Firmware files:
    - When the device reboots, the bootloader checks for the presence of firmware files, and flashes them (if valid)

USB Protocol Reversing

- The basics of the protocol were already reversed:
  - github.com/ryanbinns/ttwatch

- My fork: github.com/lgrangeia/ttwatch
  - '**ttdiag**' tool sends raw USB commands
  - Parses USBPCap files (captured from the oficial Windows software)

USB Protocol Reversing

- Protocol Format (this one formats the EEPROM)
  - 09 | Size | Seq | CMD| [arguments]

```
OUT:   09 02 D1 0E
IN:    01 16 D1 0E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

- 255 possible commands (some with arguments)
- Fuzzing found interesting stuff (about 60 different commands):
  - Test menu
  - Diagnostic messages
  - Screenshots
  - etc.
- Interesting attack surface, might yield results later.

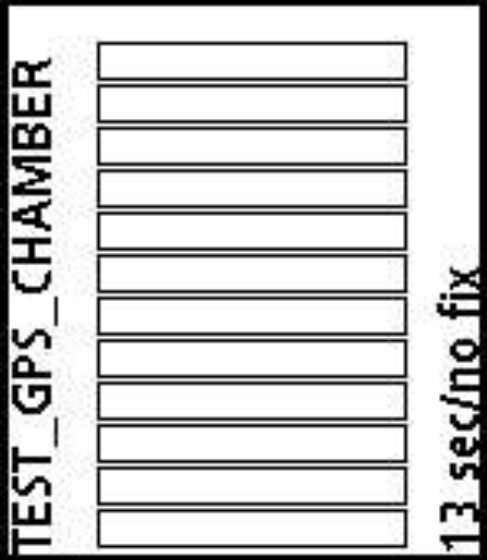
```
## Response message "01 06 17 28 00 00 00 18" appears to be
## "unused command"

0x00        # unused
0x01        # unused
0x02        # open file for writing
0x03        # delete file
0x04        # write file data
0x05        # get file size
0x06        # open file for reading
0x07        # read file data request
0x08        # unknown (appears file related)
0x09        # read file data response
0x0a        # find close file
0x0b        # unused
0x0c        # close file
0x0d        # creates a new config file (0x00f20000) if one doesn't exist
0x0e        # *** WARNING*** clears eeprom
0x0f        # returns 20 null bytes
0x10        # resets device, returns data
0x11        # find first file
0x12        # find next file
0x13        # unused
0x14        # get current time
0x15        # appears also time related (date?)
0x16        # returns 4 null bytes
0x17        # acks, returns nothing
0x18        # Enters test mode! menu!
                    # Test mode commands:
                    # down:     test gps open sky
                    # up:                 test gps chamber
                    # left:               test_ttff
                    # right:    ohr_read_sensor

0x19        # Appears to exit test mode, resets device
0x1a        # unknown (used), returns 4 bytes of data (battery?)
0x1b        # returns 12 null bytes
0x1c        # write to lcd controller
0x1d        # reset gps processor "wait 1 minute before disconnecting USB"
0x1e        # "Reset GSD done!"
0x1f        # returns 4 byte null
0x20        # get product id
0x21        # get firmware version
0x22        # returns 1 byte null
0x23        # returns 65 00 00 or 66 00 00 (some levels?)
0x24        # returns 4 byte null
0x25        # take screenshot! 0x0086000n
0x26        # no data
0x27        # unused
0x28        # get ble firmware version
0x29        # "jenkins-berlin-rcl 68 2015-06-18_10-43-37"
0x2a        # "undef"
0x2b        # "RCL"
0x2c        # "1969105"
0x2d        # no data
0x2e        # 00 3C 28 7A 00 00 01 67 00 00 00 73 00 00 00 25 00 00 00 00
                    # these could be registers:
0x2f        # seems to accept 4 byte argument, flag register? 00 00 3F 95
0x30        # 00 02 80 00
0x31        # appears to accept 4 byte argument, last byte is response size (good to fuzz)
0x32        # unused
0x33        # 12 bytes return
0x34        # 12 bytes return, also dependant of argument (good to fuzz)

0x35        # "nothing to report" <- could be error logs
0x36        # unused...
```

Ready for cmd
TOMTOM
Waiting

ACCEL
2304
TomTom
-3328
64

screenshot3

Previous Next



144 × 168 pixels 3,0 kB 100% 3 / 4

EEPROM File structure

- Device contains 4MB EEPROM with a primitive filesystem
- Each file can be read or written to via USB (and bluetooth)
- Name structure is 32 bit values
- Coincident with firmware files

EEPROM File structure

```
lgrangeia@vostok: ~
lgrangeia@vostok:~$ ttwatch -l
0x00810004: 4198
0x00810005: 4136
0x00810009: 3968
0x0081000b: 3980
0x0081000a: 4152
0x0081000e: 3957
0x0081000d: 4189
0x0081000f: 4156
0x0081000c: 4003
0x00810002: 4115
0x00010200: 43532
0x00710701: 74
0x00710700: 74
0x00710004: 140
0x00710703: 98
0x00710702: 98
0x00710704: 140
0x00010100: 28433
0x00810001: 3795
0x0002000f: 20
0x00850001: 1990
0x00810008: 6009
0x00850000: 1990
0x00810003: 3795
0x00013000: 2304
0x00020001: 72
0x00710000: 98
0x00710002: 74
0x00710001: 74
0x00710003: 98
0x00020005: 420
0x00f20000: 844
0x00000024: 4
0x00860000: 3047
0x00860001: 3047
0x00860002: 3047
0x00860003: 3047
0x00000025: 4
lgrangeia@vostok:~$
```

```
lgrangeia@vostok: ~
0x00860002: 3047
0x00860003: 3047
0x00000025: 4
lgrangeia@vostok:~$ ttwatch -r  0x00020001
P15xp__5.5.116-R32+5xpt_5.5.116-R32EGSD5xplgrangeia@vostok:~$
lgrangeia@vostok:~$ ttwatch --delete 0x00020001
lgrangeia@vostok:~$ ttwatch -r  0x00020001
Unable to read file
lgrangeia@vostok:~$ echo 123 | ttwatch -w 0x00020001
lgrangeia@vostok:~$ ttwatch -r  0x00020001
123
lgrangeia@vostok:~$
```

Exploitation

Status

- We have a fairly large external attack surface:
  - USB
  - EEPROM Files
    - read / write all files
  - Bluetooth (fairly unexplored)

Problems

- We have **zero knowledge** of the operating environment
  - Address Mappings / Threads / Libs / Interrupts??

Challenges

- Get our code running on the device
- First try:
  - Fuzzing the language files
  - ASCII strings → Buffers → overflow?

Language Files

```
00000000   00 01 02 03 04 05 06 07  08 09 0a 0b 0c 0d 0e 0f
00000000   cb 0e 00 00 b3 01 00 00  4d 6f 6e 00 54 75 65 00   ....³...Mon.Tue.
00000010   57 65 64 00 54 68 75 00  46 72 69 00 53 61 74 00   Wed.Thu.Fri.Sat.
00000020   53 75 6e 00 4d 4f 4e 00  54 55 45 00 57 45 44 00   Sun.MON.TUE.WED.
00000030   54 48 55 00 46 52 49 00  53 41 54 00 53 55 4e 00   THU.FRI.SAT.SUN.
00000040   4d 4f 4e 44 41 59 00 54  55 45 53 44 41 59 00 57   MONDAY.TUESDAY.W
00000050   45 44 4e 45 53 44 41 59  00 54 48 55 52 53 44 41   EDNESDAY.THURSDA
00000060   59 00 46 52 49 44 41 59  00 53 41 54 55 52 44 41   Y.FRIDAY.SATURDA
00000070   59 00 53 55 4e 44 41 59  00 4a 41 4e 00 46 45 42   Y.SUNDAY.JAN.FEB
00000080   00 4d 41 52 00 41 50 52  00 4d 41 59 00 4a 55 4e   .MAR.APR.MAY.JUN
00000090   00 4a 55 4c 00 41 55 47  00 53 45 50 00 4f 43 54   .JUL.AUG.SEP.OCT
000000a0   00 4e 4f 56 00 44 45 43  00 4d 00 54 00 57 00 54   .NOV.DEC.M.T.W.T
000000b0   00 46 00 53 00 53 00 4f  54 48 45 52 00 53 54 4f   .F.S.S.OTHER.STO
000000c0   50 57 41 54 43 48 00 48  49 53 54 4f 52 59 00 53   PWATCH.HISTORY.S
000000d0   45 54 54 49 4e 47 53 00  52 55 4e 00 43 59 43 4c   ETTINGS.RUN.CYCL
000000e0   45 00 53 57 49 4d 00 53  57 49 4d 00 4c 41 50 00   E.SWIM.SWIM.LAP.
000000f0   43 55 52 52 45 4e 54 00  43 4c 4f 43 4b 00 53 4f   CURRENT.CLOCK.SO
00000100   55 4e 44 00 50 52 4f 46  49 4c 45 00 4c 41 4e 47   UND.PROFILE.LANG
00000110   55 41 47 45 00 57 41 54  43 48 20 53 54 41 54 55   UAGE.WATCH STATU
00000120   53 00 61 6c 61 72 6d 00  6f 6e 00 6f 66 66 00 72   S.alarm.on.off.r
00000130   65 61 64 79 00 41 56 47  2e 20 50 41 43 45 00 41   eady.AVG. PACE.A
00000140   56 47 2e 53 50 45 45 44  00 41 56 47 2e 44 49 53   VG.SPEED.AVG.DIS
```

- List of NUL terminated ASCII strings
- First 4 bytes: length of all strings inc. nulls (little endian)
- Next 4 bytes: number of strings (little endian)

Language Files

- Tested printf() style format strings
- Tested zero length strings
- Tested oversized files

Zero length language file (video)

- Empty strings, non-empty number of strings
- Result: Partial look into stack region?

Big language file (video)

- String buffer over 6000 bytes
- Result: Crash!

Language files

- Device resets (interesting)…
- A mistery file appears (0x00013000)…

First crash!

01.crash

```
Crashlog - SW ver 1.8.42
 R0 = 0x2001b088
 R1 = 0x2001bf54
 R2 = 0x2001cee4
 R3 = 0x2001cdd4
 R12 = 0x0045bf51
 LR [R14] = 0x00426a75 subroutine call return address
 PC [R15] = 0x2001bf54 program counter
 PSR = 0x20000000
 BFAR = 0xe000ed38
 CFSR = 0x00020000
 HFSR = 0x40000000
 DFSR = 0x00000000
 AFSR = 0x00000000
 Batt = 4200 mV
 stack hi = 0x00000514
ÿÿÿÿÿÿÿÿÿÿÿÿ
```

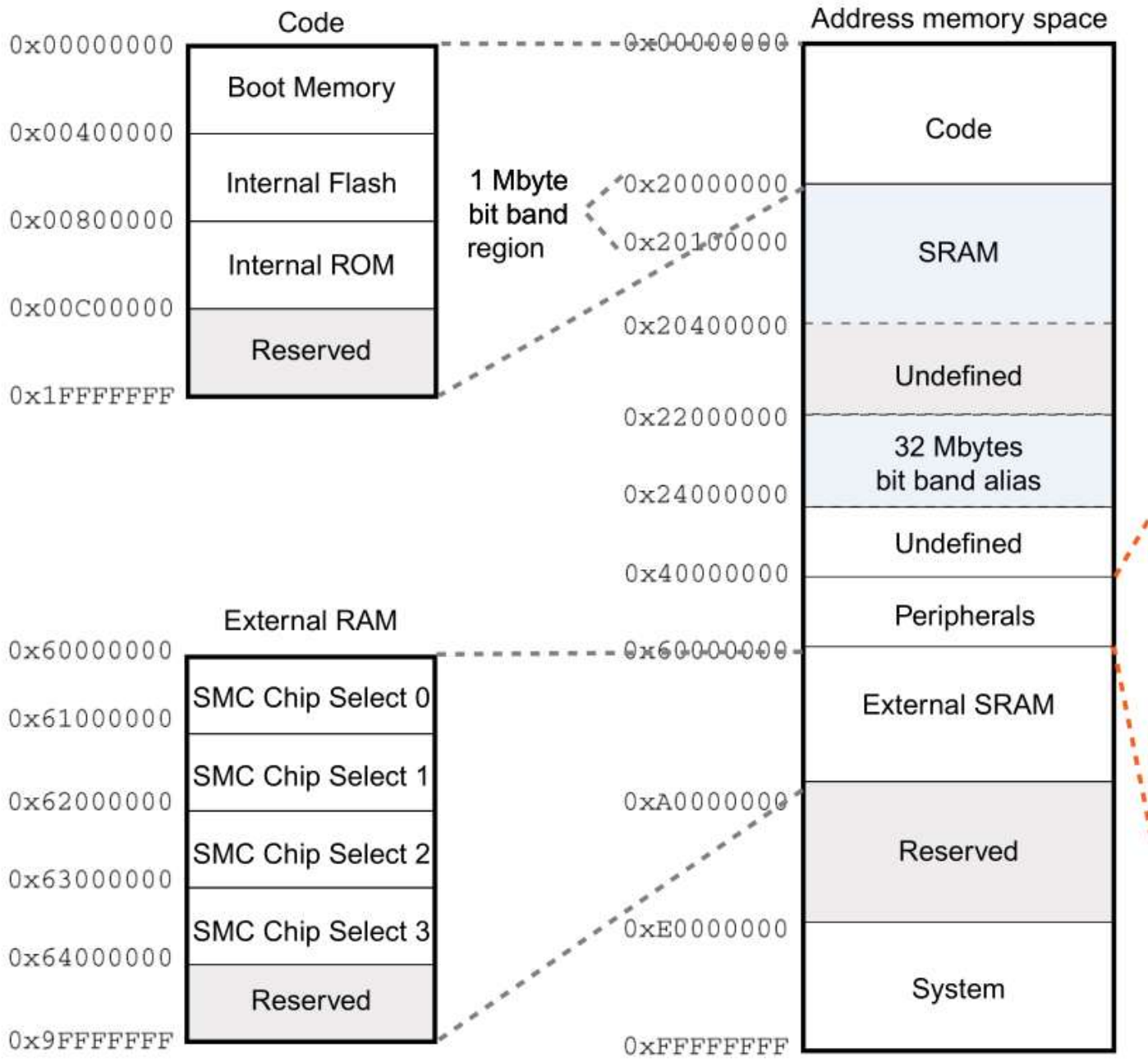
Line 13, Column 19 | Tab Size: 4 | Plain Text

Address Space

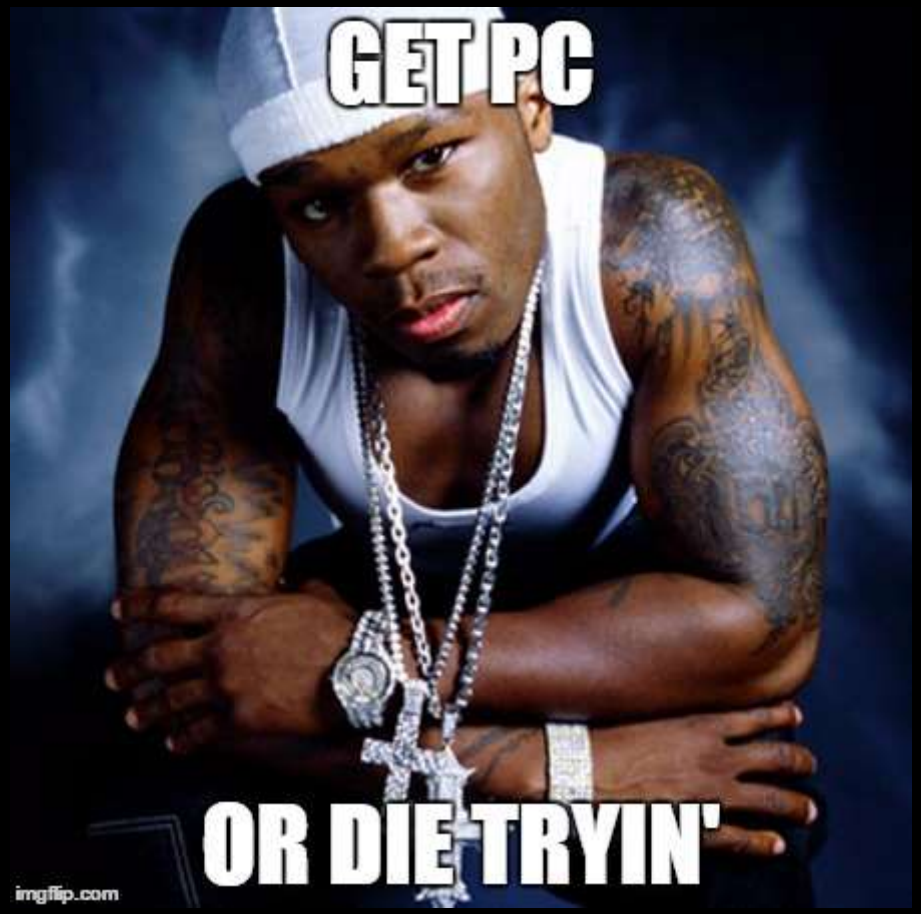
- Collected a LOT of crash dumps
- Read (and must read more) ARM documentation
- Note:
  - This is an ARM Cortex M4 CPU
  - Works in ARM Thumb-32 mode exclusively
  - No ASLR (predictable)
  - Not many memory controls (SRAM is executable)

- LR [R14] = 0x00426a75 subroutine call return address
- PC [R15] = 0x2001bf54 program counter
- We're in SRAM (heap or stack)
- Return address is in Flash region
- **Nice.**

Figure 7-1. SAM4S Product Mapping



After some fiddling...



R0 = 0x00000000
R1 = 0xffffffe3
R2 = 0x00000002
R3 = 0x00000029
R12 = 0x00000000
LR [R14] = 0x00441939 subroutine call return address
PC [R15] = 0x000000cc program counter

Arbitrary code execution

Yep, reliable arbitrary, code execution ***on my wrist***!

```
.thumb

mov r2, #0x13
mov r3, #0x37

add r1, r3, r2, lsl #8
```

Crashlog - SW ver 1.8.42
R0 = 0x00000000
R1 = 0x00001337
R2 = 0x00000013
R3 = 0x00000037
R12 = 0x00000000
LR [R14] = 0x00441939 subroutine call return address
PC [R15] = 0x2001a48c program counter

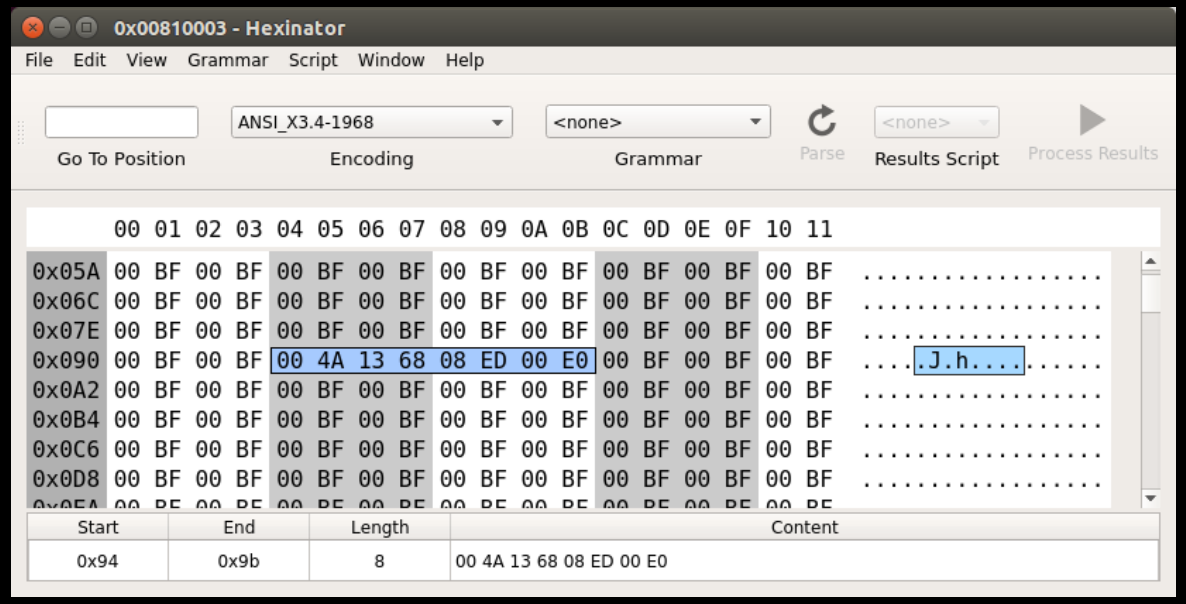
Not quite there yet

- Still not sure where we are exactly in SRAM
  - Most likely heap segment (stack is too far below)
  - We can return to the calling subroutine and resume execution
  - No clue about stack status
- But no clue where else we can jump to/call
  - Wild guess: derivative of FreeRTOS?
- Only door to the outside world is crash file.
- Main goal: obtain AES key to decrypt firmware and “turn the lights on”

Currently...

- Still figuring out address space

```
lgrangeia@vostok:~/tomtom/Overflow1/testing4$ arm-none-eabi-as -mcpu=cortex-m4 -o getvtor.o getvtor.a
lgrangeia@vostok:~/tomtom/Overflow1/testing4$ arm-none-eabi-objdump -d getvtor.o

getvtor.o:     file format elf32-littlearm


Disassembly of section .text:

00000000 <.text>:
   0:   4a00            ldr     r2, [pc, #0]    ; (4 <.text+0x4>)
   2:   6813            ldr     r3, [r2, #0]
   4:   e000ed08        .word   0xe000ed08
lgrangeia@vostok:~/tomtom/Overflow1/testing4$
```



Next Steps

Next steps

- Achieve efficient data exfiltration
  - AES key may be extractable from bootloader
  - Possible exfiltration vectors:
    - Menu strings
    - File operations / Hardfault handler
    - System calls
- Once firmware key is found, custom firmware is possible

Next steps

- Explore same issues via Bluetooth
  - Watch is usable during bluetooth connection
  - Most USB commands appear to work via Bluetooth
    - github.com/dlenski/ttblue
  - **Lots** of potential here

Help needed!

- Do you have a tomtom runner/multisport/cardio?
  - Can you borrow one?
- Do you like reversing?
- Do you know ARM Thumb-2 instruction set?
- Do you like to read hardware datasheets?

Join us!

(there will be beer)

Takeaways

- IoT is friendly for noob reversers
  - Cutting edge exploit development for x86/x64? Not so much
- This is not earth shattering end of world vulnerability
  - Get your risk assessment on (hackers usually suck at it)
- IoT security is weak. Real consequences / deaths for some fields:
  - (self driving) cars, airplanes, pacemakers, etc.
- Think twice before connecting your car/airplane to the internet.

Thanks!
Questions?

Luis Grangeia | @lgrangeia

September 2015
Confraria de Segurança da Informação